

JavaScript Notes — Day 3

Chapter 3: Loops and Strings

for Loop

Repeats a block a fixed number of times, using a counter with an init, condition, and increment.

```
for (let i = 0; i < 5; i++) {  
  console.log("chiru");  
}  
  
for (let count = 1; count <= 10; count++) {  
  console.log(count);  
}
```

Example — sum of first 10 natural numbers

```
let sum = 0;  
for (let i = 1; i <= 10; i++) {  
  sum += i; // sum = sum + i  
  console.log(sum);  
}
```

while Loop

Repeats a block as long as a condition stays true. The counter is managed manually.

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

for...of Loop

Iterates over the values of an iterable, such as a string or array — here, each character of a string.

```
let str = "chiru";  
for (let ch of str) {  
  console.log("ch=", ch); // c, h, i, r, u  
}
```

Example — counting string length manually

```
let size = 0;  
for (let ch of str) {  
  size++;  
}  
console.log(size);
```

Note: to just get the length, JavaScript's built-in **str.length** property does this directly — no loop needed.

for...in Loop

Iterates over the enumerable property keys of an object.

```
let stu = {
  name: "goat",
  age: 23,
  cgpa: 9.1,
  isPass: true
};

for (let key in stu) {
  console.log(key); // name, age, cgpa, isPass
}
```

Note: object properties use a colon (key: value), not an equals sign — `age = 23` would be a syntax error inside an object literal.

Practice Problems — Loops

1. Print even numbers from 0 to 100

```
for (let n = 0; n <= 100; n++) {
  if (n % 2 === 0) {
    console.log("num=", n);
  }
}
```

2. Number-guessing game

```
let guess = 45;
let userInput = prompt("enter a number:");

while (Number(userInput) !== guess) {
  userInput = prompt("wrong, enter again");
}
console.log("perfect, YOU WON");
```

Note: `prompt()` returns a string, so compare with `Number(userInput)` or use `==` to avoid type mismatches.

Strings

Accessing characters and length

```
let name = "chiru";
console.log(name[2]); // "i"
console.log(name.length); // 5 (a property, not a function)
```

Template Literals

Use backticks (` `) to embed expressions with `${ }` directly inside a string.

```
let tl = `the goat of cricket is vk`;

let obj = { item: "pen", price: 10 };
let out = `the cost of ${obj.item} is ${obj.price} rupees`;
console.log(out);
```

Escape sequences: `\n` = new line, `\t` = new tab.

Common String Methods

Method	Description
<code>toUpperCase()</code>	Converts the string to uppercase
<code>toLowerCase()</code>	Converts the string to lowercase
<code>trim()</code>	Removes whitespace from both ends
<code>slice(start, end)</code>	Extracts a section of the string
<code>concat(str2)</code>	Joins two or more strings
<code>replace(old, new)</code>	Replaces a substring with another (quotes needed around text)
<code>charAt(index)</code>	Returns the character at the given index

```
let str = "chiru the developer ";
let str2 = "and cr, vk18 also";

str.toUpperCase();
str.toLowerCase();
str.trim(); // removes extra spaces
str.slice(1, 6);
str.concat(str2);
str.replace("chiru", "chiranth"); // arguments need quotes
str.charAt(5);
```

Note: string methods don't change the original string — they return a new one. Assign the result if you want to keep it, e.g. `str = str.toUpperCase()`;

Practice Problem — Strings

Prompt the user for their full name, then output it as `@name + name length`.

```
let name = prompt("enter your name :");
let user = "@" + name + name.length;
console.log(user);
```